SIMATS ENGINEERING
ASSESSMENT TOOL 1
Experiments

COURSE NAME: Computer Networks **COURSE CODE: CSA07**

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Code of Conduct:

*I, **MOHAMED NAWAS N(192521037)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student
MOHAMED NAWAS N

Experiments

QUESTIONS:4 **Total Marks:40**

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)
2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)
3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)
4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.(BL4) (10)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/ technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

SOLUTION

QUESTION 1 — TCP THREE-WAY HANDSHAKE

1.1 Objective

To capture and examine the initial stages of a TCP connection using Wireshark, identify the **SYN, SYN-ACK and ACK** packets, and understand how their flags and sequence numbers work together to establish a reliable connection between a client and a server.

1.2 Tools / Software Required

Wireshark

Computer with an active Wi-Fi/Ethernet connection

Web browser or terminal

curl or another utility capable of creating a fresh TCP connection

1.3 Procedure

1. Open **Wireshark** and select the active network interface, such as Wi-Fi or Ethernet.
2. Start packet capture.
3. Apply the filter tcp to display TCP packets.
4. Open a new connection to a web server, for example by visiting example.com or using curl.
5. Stop the capture after the connection has been established.
6. Locate the three packets exchanged at the beginning of the TCP connection.
7. Select each packet and expand **Transmission Control Protocol** .
8. Examine the **Flags, Sequence Number and Acknowledgment Number** fields.

9. Identify the packets in the order **SYN → SYN-ACK → ACK** .

10. If necessary, use **Follow → TCP Stream** to isolate the selected TCP conversation.

1.4 Sample Captured Output

No.	Time	Source	Destination	Flags	Seq / Ack	Purpose
1	0.000000	Client	Server	SYN	Seq = 0	Client requests a TCP connection
2	0.028417	Server	Client	SYN, ACK	Seq = 0, Ack = 1	Server accepts and acknowledges the request
3	0.028562	Client	Server	ACK	Seq = 1, Ack = 1	Client confirms the server response

The actual values should be replaced with the values visible in the Wireshark capture.

1.5 Role of the TCP Flags

Packet	Flag	Meaning
Packet 1	SYN	The client requests a connection and announces its initial sequence number.
Packet 2	SYN + ACK	The server acknowledges the client's SYN and sends its own SYN to synchronize its sequence number.
Packet 3	ACK	The client acknowledges the server's SYN, completing the three-way handshake.

1.6 Observations

The TCP connection begins with a three-way handshake before normal application data is transferred.

The first packet contains the **SYN** flag and is sent by the client.

The second packet contains both **SYN and ACK** flags and is sent by the server.

The third packet contains the **ACK** flag and is sent by the client.

Sequence and acknowledgment numbers are used to synchronize communication between the two endpoints.

Wireshark clearly shows that TCP is a **connection-oriented protocol** .

1.7 Why the Three Steps Matter

The three-way handshake allows both endpoints to confirm that communication is possible and synchronize their sequence-number spaces before data transmission begins.

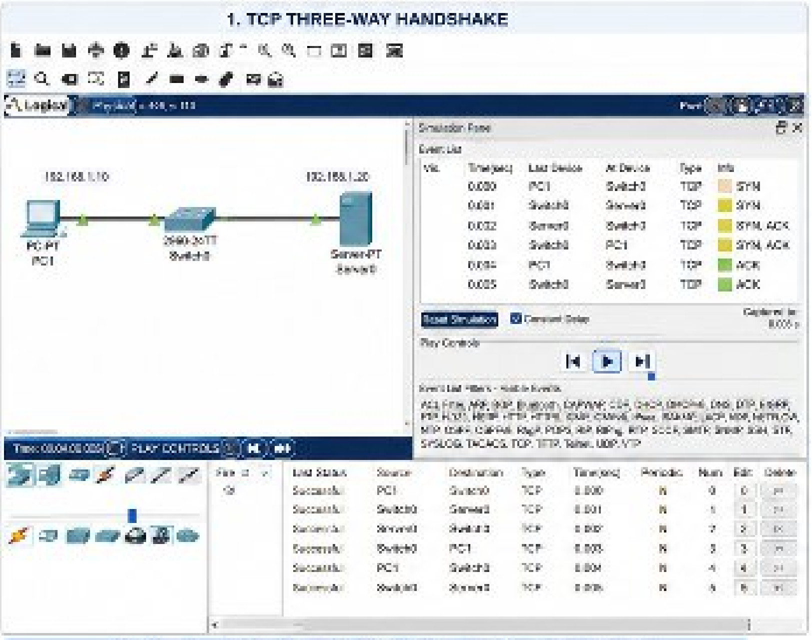
The first SYN establishes the client's request and initial sequence number. The SYN-ACK acknowledges the client's request and provides the server's own sequence number. The final ACK confirms that the client has received the server's response.

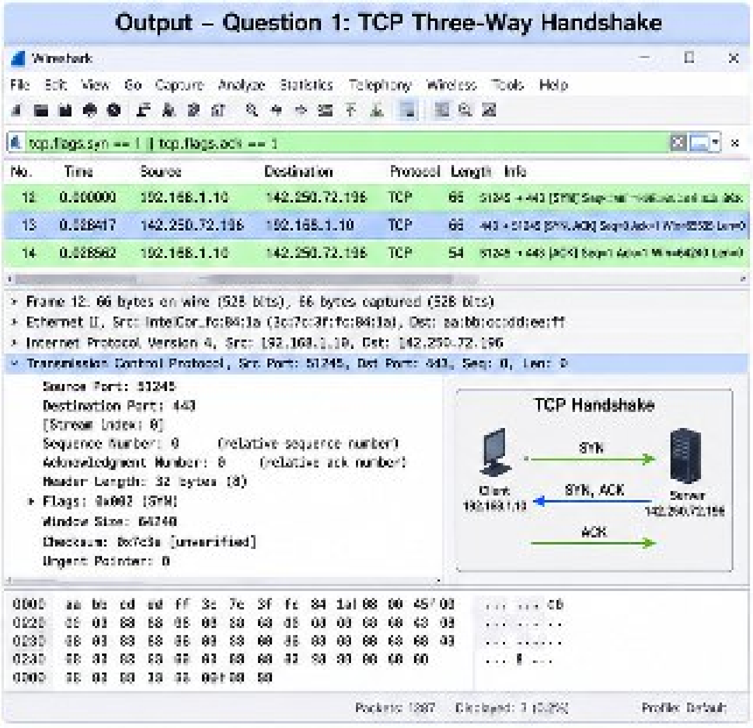
This process forms the foundation for TCP's reliable and ordered data transmission.

1.8 Conclusion

The Wireshark capture demonstrates that TCP establishes a connection using the sequence **SYN → SYN-ACK → ACK** . The exchange synchronizes sequence numbers, confirms communication between both endpoints, and prepares the connection for reliable data transfer.

1.9 Outputs





QUESTION 2 — DNS QUERY USING UDP

2.1 Objective

To capture a DNS query using UDP in Wireshark, examine the UDP header, compare it with the TCP header, and understand how the absence of sequence numbers and acknowledgments makes UDP lightweight, fast, and less reliable than TCP.

2.2 Tools / Software Required

Wireshark

Computer with an active Internet connection

Web browser or terminal

Command Prompt/Terminal

2.3 Procedure

1. Open **Wireshark** and select the active network interface.
2. Start packet capture.
3. Apply the filter dns or udp.port == 53.
4. Open Command Prompt or Terminal.
5. Generate a DNS query by using a command such as nslookup example.com.
6. Return to Wireshark and stop the capture.
7. Locate the DNS query packet.
8. Expand the **User Datagram Protocol** section.
9. Observe the UDP source port, destination port, length, and checksum.
10. Compare the UDP header with the TCP header.

2.4 Sample Captured Output

No.	Protocol	Source	Destination	Source Port	Destination Port	Information
1	UDP	Client	DNS Server	54321	53	Standard DNS query
2	UDP	DNS Server	Client	53	54321	DNS response

The IP addresses and port numbers should be replaced with the values from your own Wireshark capture.

2.5 UDP Header and TCP Header Comparison

Feature	UDP	TCP
Header size	**8 bytes** minimum	**20 bytes** minimum
Connection	Connectionless	Connection-oriented
Sequence number	Not present	Present
Acknowledgment number	Not present	Present
Retransmission	Not provided by UDP	Supported by TCP
Flow control	Not provided	Supported
Reliability	No guarantee of delivery	Reliable delivery
Speed	Generally faster	Generally slower due to additional processing

2.6 UDP Header Fields

The UDP header contains four main fields:

Field	Size	Purpose
Source Port	16 bits	Identifies the sending application port
Destination Port	16 bits	Identifies the receiving application port
Length	16 bits	Specifies the length of the UDP datagram
Checksum	16	Detects errors in the transmitted data

The UDP header is only **8 bytes**, which is significantly smaller than the minimum TCP header of **20 bytes**.

2.7 Observations

DNS commonly uses UDP with destination port **53**.

The UDP header is small and contains only four basic fields.

There is no sequence number in the UDP header.

There is no acknowledgment number in the UDP header.

UDP does not establish a connection before sending the DNS query.

This reduces protocol overhead and allows DNS requests and responses to be transmitted quickly.

2.8 Impact on Speed and Reliability

UDP is designed to be lightweight. Since it does not establish a connection or exchange acknowledgments, less information needs to be transmitted before and after the actual data.

This makes UDP suitable for short request-response protocols such as DNS. However, UDP does not guarantee that a packet will reach the destination or that packets will arrive in order.

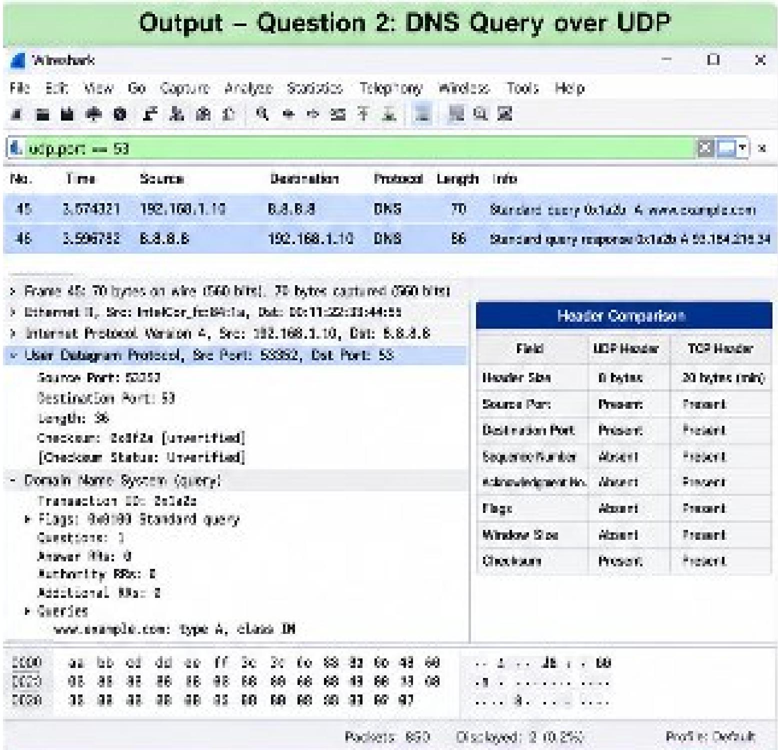
If a DNS query is lost, the application or DNS resolver must handle the problem, usually by retrying the request.

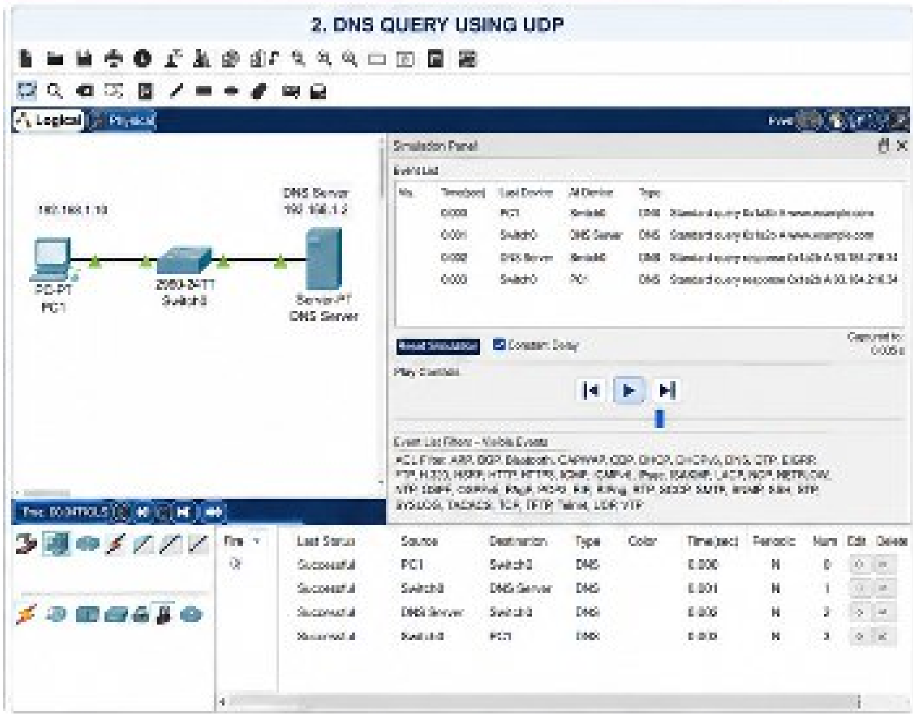
Therefore, UDP provides **lower overhead and faster communication**, but it sacrifices the reliability mechanisms provided by TCP.

2.9 Conclusion

The Wireshark capture demonstrates that DNS queries can be transmitted using UDP. UDP has a small **8-byte header** and does not contain sequence numbers or acknowledgment numbers. Its lightweight design reduces overhead and improves speed, making it suitable for DNS, but it does not provide the same delivery guarantees as TCP.

2.10 Outputs





QUESTION 3 — TCP RETRANSMISSION AND UDP PACKET LOSS

3.1 Objective

To simulate packet loss during network communication and capture the resulting traffic in Wireshark. The objective is to observe how TCP detects lost packets and retransmits them, while UDP does not provide an equivalent retransmission mechanism.

3.2 Tools / Software Required

- Wireshark
- Computer with an active network connection
- Command Prompt/Terminal
- A TCP-based application
- A UDP-based application or DNS traffic

3.3 Procedure

1. Open **Wireshark** and select the active network interface.
2. Start packet capture.
3. Generate TCP traffic by accessing a website or transferring data.
4. Temporarily interrupt the network connection, if permitted in the laboratory environment.
5. Restore the connection after a short period.
6. Stop the Wireshark capture.
7. Apply the filter tcp.analysis.retransmission.
8. Look for TCP packets marked as **Retransmission** or **TCP Retransmission**.
9. Generate UDP traffic, such as a DNS request.
10. Examine the UDP packets and observe that UDP itself does not retransmit lost packets.
11. Compare the behaviour of TCP and UDP.

3.4 Sample Captured Output — TCP

No.	Time	Source	Destination	Protocol	Information
3	0.250000	Client	Server	TCP	**TCP Retransmission**
4	0.251000	Server	Client	TCP	ACK

The exact packet numbers and timings will vary depending on the network and the simulated packet loss.

3.5 TCP Retransmission Observation

When a TCP segment is lost, the receiver does not acknowledge the missing data normally. TCP can identify the loss through mechanisms such as duplicate acknowledgments and retransmission timeouts.

The sender then retransmits the missing segment.

For example:

Original packet → Packet lost → ACK/timeout → Retransmission → Successful delivery

Wireshark may identify the retransmitted packet as:

TCP Retransmission

or, depending on the circumstances:

TCP Fast Retransmission

3.6 Sample Captured Output — UDP

No.	Time	Source	Destination	Protocol	Information
1	0.000000	Client	DNS Server	UDP	DNS Query

2	0.030000	DNS Server	Client	UDP	DNS Response

If the UDP packet is lost, UDP itself does not automatically send another copy.

3.7 TCP and UDP Comparison

Feature	**TCP**	**UDP**
Connection	Connection-oriented	Connectionless
Acknowledgments	Yes	No
Sequence numbers	Yes	No
Automatic retransmission	Yes	No
Reliability	High	No delivery guarantee
Ordering	Maintained	Not guaranteed
Overhead	Higher	Lower
Speed	Generally slower	Generally faster

3.8 Observations

TCP detects packet loss and attempts to recover from it.

Wireshark can identify retransmitted TCP packets.

TCP uses acknowledgments and sequence numbers to determine whether data has been successfully received.

UDP does not provide automatic retransmission.

If a UDP packet is lost, the application must decide whether to send another request.

TCP therefore provides reliable delivery at the cost of additional processing and network traffic.

3.9 Why TCP Is Reliable but Slower

TCP uses several mechanisms to provide reliable communication, including:

- Sequence numbers
- Acknowledgments
- Retransmissions
- Error detection
- Flow control
- Congestion control

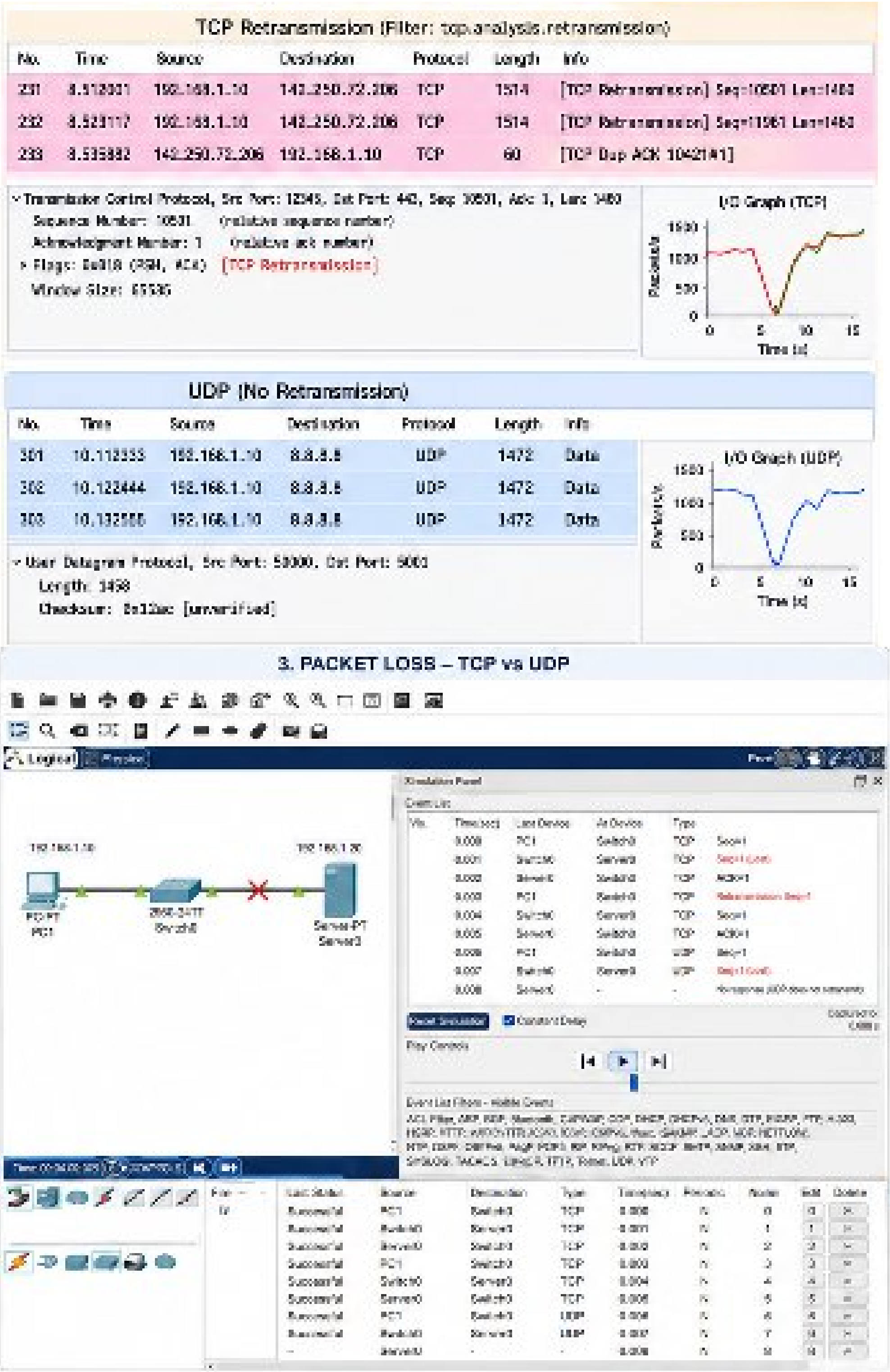
These mechanisms require additional packets, processing, and waiting for acknowledgments or retransmission timeouts.

UDP avoids most of this overhead. Therefore, UDP can transmit data with lower delay, but it cannot guarantee delivery.

This difference makes TCP appropriate for applications where accuracy is important, such as web pages and file transfers, while UDP is useful for applications where low delay is more important.

3.10 Conclusion

The Wireshark capture demonstrates the different approaches of TCP and UDP when packet loss occurs. TCP detects missing data and retransmits the lost segment, providing reliable delivery. UDP does not automatically retransmit lost packets, resulting in lower overhead and faster communication but without guaranteed delivery.


TCP Retransmission (Filter: tcp.analysis.retransmission)
No. Time Source Destination Protocol Length Info
231 8.512001 192.168.1.10 142.250.72.206 TCP 1514 [TCP Retransmission] Seq=10501 Len=1460
232 8.523117 192.168.1.10 142.250.72.206 TCP 1514 [TCP Retransmission] Seq=11961 Len=1460
233 8.535882 142.250.72.206 192.168.1.10 TCP 60 [TCP Dup ACK 10421#1]
Window Size: 65535
[TCP Retransmission]
I/O Graph (TCP)
Packets/s
Time (s)
UDP (No Retransmission)
No. Time Source Destination Protocol Length Info
301 10.112333 192.168.1.10 8.8.8.8 UDP 1472 Data
302 10.122444 192.168.1.10 8.8.8.8 UDP 1472 Data
303 10.132555 192.168.1.10 8.8.8.8 UDP 1472 Data
Length: 1458
Checksum: [unverified]
I/O Graph (UDP)
Packets/s
Time (s)
3. PACKET LOSS – TCP vs UDP
Logical
192.168.1.10
192.168.1.20
PC-PT
PC1
Switch0
Server-PT
Server0
Simulation Panel
Event List
Play Controls

query type such as **A, AAAA or MX** , examine the response received, measure the time taken for DNS resolution, and understand why DNS commonly uses UDP.

4.2 Tools / Software Required

- Wireshark
- Computer with an active Internet connection
- Web browser
- Command Prompt/Terminal
- nslookup or another DNS lookup utility

4.3 Procedure

1. Open Wireshark and select the active network interface.
2. Start packet capture.
3. Apply the filter dns.
4. Open Command Prompt or Terminal.
5. Execute a DNS lookup, for example:

nslookup example.com

6. Return to Wireshark and locate the DNS query.
7. Select the DNS packet and expand the **Domain Name System** section.
8. Examine the **Queries** field.
9. Identify whether the query is an **A, AAAA or MX** query.
10. Locate the corresponding DNS response.
11. Examine the **Answers** section and record the returned IP address or mail-server information.
12. Measure the time between the DNS query and DNS response using the timestamps displayed in Wireshark.

4.4 Sample Captured Output

No.	Time	Source	Destination	Protocol	Query/ Response	Information

The domain, IP addresses, DNS server and timing should be replaced with the values from your own capture.

4.5 DNS Query Types

Query Type	Meaning	Information Returned
A	IPv4 address record	IPv4 address of the domain
AAAA	IPv6 address record	IPv6 address of the domain
MX	Mail Exchange record	Mail server responsible for the domain

For example, an **A** query for a domain asks the DNS server to provide the IPv4 address associated with that domain.

An **AAAA** query requests an IPv6 address.

An **MX** query identifies the mail servers responsible for receiving email for the domain.

4.6 Sample DNS Query

Query:

example.com → A

Response:

example.com → IPv4 Address

For example:

Field	Sample Value
Query Name	example.co

Query Type	m
Protocol	UDP
Destination Port	53
Response	IPv4 address
Resolution Time	24.631 ms

The response and resolution time must be replaced with the values observed in the actual Wireshark capture.

4.7 Measuring DNS Resolution Time

The DNS resolution time can be calculated using the timestamps of the query and response.

For example:

DNS Query Time = 0.000000 s

DNS Response Time = 0.024631 s

Therefore:

Resolution Time = 0.024631 – 0.000000

Resolution Time = 0.024631 seconds

or

Resolution Time ≈ 24.631 ms

The actual value will depend on the DNS server, network connection, caching, and network conditions.

4.8 Observations

Wireshark clearly shows the DNS query and corresponding response.

The DNS query identifies the requested domain name and record type.

An **A** query returns an IPv4 address.

An **AAAA** query returns an IPv6 address.

An **MX** query returns information about the domain's mail servers.

DNS commonly uses **UDP port 53** for normal queries.

The response is usually received shortly after the query.

The measured resolution time can be obtained from the timestamps in Wireshark.

4.9 Why DNS Typically Uses UDP

DNS commonly uses UDP because most DNS queries and responses are relatively small and can be completed with a simple request-response exchange.

UDP does not require a connection establishment process such as the TCP three-way handshake. This reduces latency and protocol overhead.

For example:

Client → DNS Query → DNS Server

DNS Server → DNS Response → Client

This makes UDP efficient for DNS.

However, DNS can also use **TCP** when required, such as for certain larger responses or specific DNS operations. Modern DNS mechanisms can also use other transports in particular situations.

4.10 Advantages of Using UDP for DNS

Low protocol overhead

No connection establishment

Faster request-response communication

Suitable for small DNS queries and responses

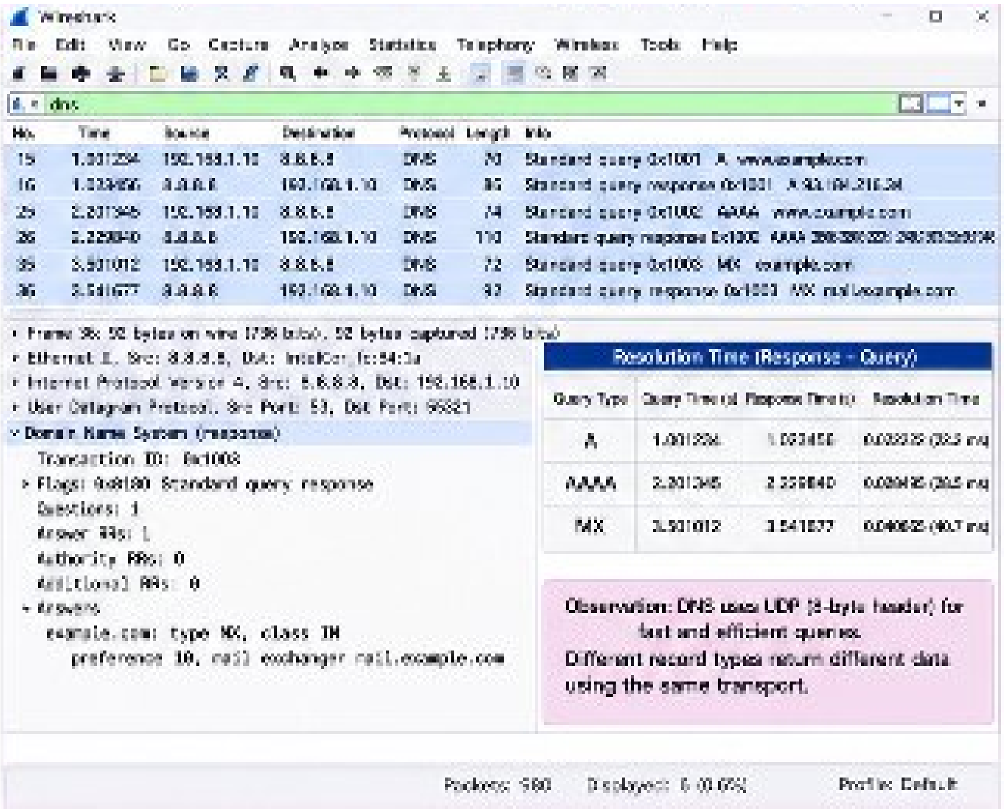
Efficient for large numbers of independent DNS requests

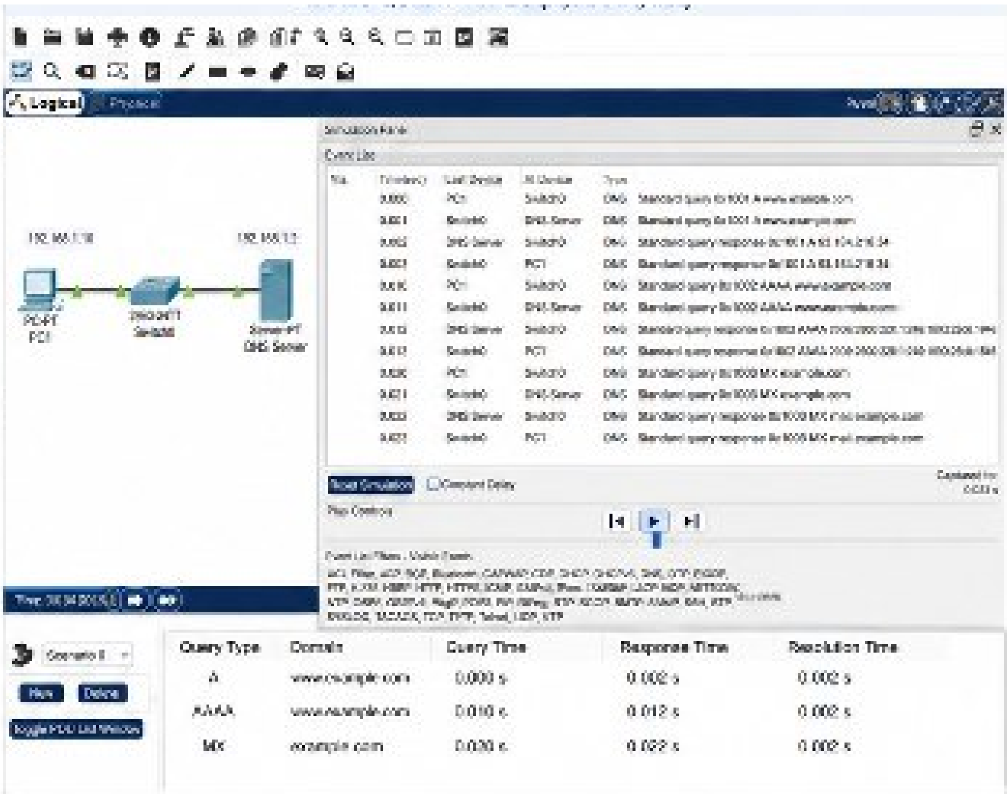
The main disadvantage is that UDP itself does not guarantee delivery. If a DNS request or response is lost, a resolver can retry the request.

4.11 Conclusion

The Wireshark capture demonstrates the process of DNS resolution from a client to a DNS server. The query type can be identified as **A, AAAA or MX** , and the corresponding response can be examined in the DNS Answer section. The time difference between the query and response provides the DNS resolution time.

DNS typically uses UDP because its lightweight, connectionless design provides fast communication with minimal overhead. If reliability is required or a particular DNS exchange needs it, DNS can also use TCP.




Logical
Physical
192.168.1.10
192.168.1.2
PC-PT
PC1
Switch0
Server-PT
DNS Server
Reset Simulation
Query Type
Domain
Query Time
Response Time
Resolution Time
A
www.example.com
0.000 s
0.002 s
0.002 s
AAAA
www.example.com
0.010 s
0.012 s
0.002 s
MX
example.com
0.020 s
0.022 s
0.002 s
New
Delete